

LAPORAN MINGGU 2 PROJECT 2 PEMROGRAMMAN PARALLEL

***SIMULASI DETEKSI HASIL PELURUHAN RADIOAKTIF U-238 MENGGUNAKAN
MONTE CARLO PARALEL***

oleh:

Ahmad Hanafi Hasan (245090300111001)

Stephen Tangguh Laia (245090300111010)

PROGRAM STUDI: S1 FISIKA



DEPARTEMEN FISIKA

FAKULTAS SAINS, TEKNOLOGI, DAN MATEMATIKA

UNIVERSITAS BRAWIJAYA

MALANG

2026

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import time

# --- 1. Parameter Simulasi ---
N_PARTICLES = 1000
energies = [4.270, 4.198] # MeV
probs = [0.79, 0.21]
detector_resolution = 0.02 # MeV

# Geometri
Z_DETECTOR = 4.0 # cm
R_DETECTOR_RING = 2.0 # cm

# Parameter Waktu Fisis (Asumsi aktivitas sumber U-238 sederhana)
ACTIVITY_BQ = 1000.0

# --- 2. Sampling Monte Carlo (Vectorized untuk N partikel) ---
start_calc_time = time.time()

# Sampling Energi
rand_p = np.random.rand(N_PARTICLES)
e_true = np.where(rand_p < probs[0], energies[0], energies[1])
e_measured = np.random.normal(e_true, detector_resolution)

# Sampling Arah (Di dalam kerucut detektor visual)
theta_max = np.arctan(R_DETECTOR_RING / Z_DETECTOR)
cos_theta = 1.0 - np.random.rand(N_PARTICLES) * (1.0 -
np.cos(theta_max))
theta = np.arccos(cos_theta)
phi = 2.0 * np.pi * np.random.rand(N_PARTICLES)

# Titik Tumbukan
r_hit = Z_DETECTOR * np.tan(theta)
x_hit = r_hit * np.cos(phi)
y_hit = r_hit * np.sin(phi)
z_hit = np.full(N_PARTICLES, Z_DETECTOR)

```

```

# Sampling Waktu Fisis (Distribusi Eksponensial untuk interval antar
peluruhan)
#  $dt = -\ln(U) / \text{Aktivitas}$ 
dt = -np.log(np.random.rand(N_PARTICLES)) / ACTIVITY_BQ
physical_time_cumulative = np.cumsum(dt)

end_calc_time = time.time()
print(f"Waktu Komputasi CPU untuk {N_PARTICLES} partikel:
{(end_calc_time - start_calc_time)*1000:.4f} milidetik")
print(f"Waktu Fisis Alamiah untuk {N_PARTICLES} partikel terdeteksi:
{physical_time_cumulative[-1]:.4f} detik")

# --- 3. Setup Visualisasi 3D ---
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

ax.set_title(f'Akumulasi Deteksi {N_PARTICLES} Partikel Alfa U-238',
fontsize=14, pad=20)
ax.set_xlabel('x (cm)')
ax.set_ylabel('y (cm)')
ax.set_zlabel('z (cm)')
ax.set_xlim([-2.5, 2.5])
ax.set_ylim([-2.5, 2.5])
ax.set_zlim([0, 4.5])

# Gambar Cincin Detektor & Sumber
theta_ring = np.linspace(0, 2 * np.pi, 100)
ax.plot(R_DETECTOR_RING * np.cos(theta_ring), R_DETECTOR_RING *
np.sin(theta_ring), np.full_like(theta_ring, Z_DETECTOR),
color='#1f2937', linewidth=2, label='Bidang detektor')
ax.scatter([0], [0], [0], color='#8b5a62', s=60, label='Sumber
U-238')

# Siapkan list objek garis dan titik (sebanyak N_PARTICLES) yang
awalnya kosong
lines = [ax.plot([], [], [], color='gray', linestyle='-',
alpha=0.3)[0] for _ in range(N_PARTICLES)]
scatters = [ax.plot([], [], [], marker='o', color='#d97706',
markersize=4, linestyle='None')[0] for _ in range(N_PARTICLES)]

```

```

info_text = ax.text2D(0.05, 0.95, "", transform=ax.transAxes,
    fontsize=11, bbox=dict(facecolor='white', alpha=0.8,
    edgecolor='gray'))
ax.legend(loc='upper right', bbox_to_anchor=(0.95, 0.9))

# --- 4. Fungsi Animasi ---
def init():
    for line, scatter in zip(lines, scatters):
        line.set_data([], [])
        line.set_3d_properties([])
        scatter.set_data([], [])
        scatter.set_3d_properties([])
    info_text.set_text("")
    return lines + scatters + [info_text]

def update(frame):
    # 'frame' merepresentasikan indeks partikel ke-i yang sedang
    # dideteksi
    i = frame

    # Munculkan garis dan titik partikel ke-i
    lines[i].set_data([0, x_hit[i]], [0, y_hit[i]])
    lines[i].set_3d_properties([0, z_hit[i]])

    scatters[i].set_data([x_hit[i]], [y_hit[i]])
    scatters[i].set_3d_properties([z_hit[i]])

    # Update teks informasi
    info_str = (f"Deteksi Partikel: {i+1} / {N_PARTICLES}\n"
        f"Energi Terakhir = {e_measured[i]:.3f} MeV\n"
        f"Waktu Fisis Berjalan =
{physical_time_cumulative[i]:.4f} detik")
    info_text.set_text(info_str)

    return lines + scatters + [info_text]

# --- 5. Eksekusi Animasi ---
# frames=N_PARTICLES berarti akan ada 50 frame, masing-masing

```

```

memunculkan 1 partikel
ani = FuncAnimation(fig, update, frames=N_PARTICLES, init_func=init,
                    blit=False, interval=200, repeat=False)

plt.tight_layout()
plt.show()

```

Digunakan jumlah partikel yang akan dideteksi hanya di nilai 1000 (seribu) partikel. Didapatkan dalam melakukan perhitungan, komputer dapat menghitungnya dalam satuan milidetik, tetapi bila dimasukkan ke bentuk visualisasi jalur distribusi tiap partikel, komputer mulai kesusahan dalam memvisualisasikannya. Hal ini dikarenakan, program harus memvisualisasikan projektil / arah pergerakan partikel satu-persatu yang akan memakan waktu lama. Dalam simulasi pertama, didapatkan waktu sekitar 4 - 6 menit untuk komputer dapat memvisualisasikan 1000 partikel yang mengenai detektor.

Validasi Fisika & Analitik

A. Parameter Energi U-238 (Ada Kesalahan Konsep)

- **Kode:** `energies = [4.270, 4.198]` dan `probs = [0.79, 0.21]`
- **Koreksi:** Untuk U-238, energi kinetik partikel alfa utamanya adalah **4.198 MeV** (probabilitas ~77.5%) dan **4.151 MeV** (probabilitas ~22.5%).

B. Geometri & Distribusi Sudut

- **Kode:** Menggunakan `cos_theta = 1.0 - np.random.rand(N_PARTICLES) * (1.0 - np.cos(theta_max))`
- **Analisis & Validasi:** Secara analitik sudah sangat akurat untuk *sampling* emisi isotropik yang dipaksa (*biased*) hanya ke dalam kerucut yang menutupi detektor. Bentuk seperti adalah teknik *variance reduction* yang ada di dalam Monte Carlo.

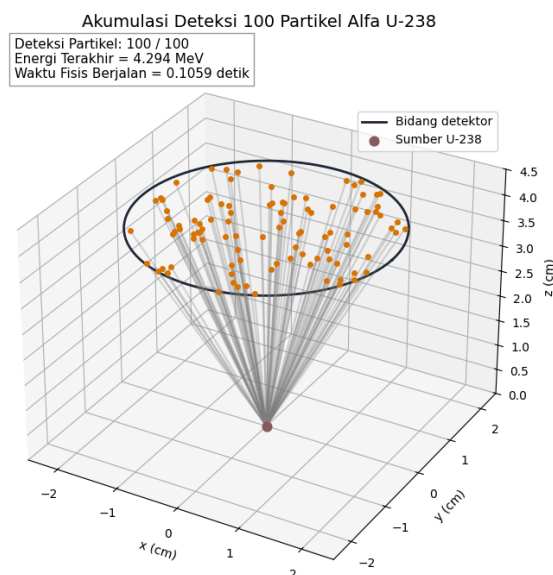
C. Perhitungan Waktu Fisis (Celah Logika Dasar)

- **Kode:** `dt = -np.log(np.random.rand(N_PARTICLES)) / ACTIVITY_BQ`
- **Analisis & Validasi:** Distribusi eksponensial Poisson yang digunakan memang rumusnya benar untuk menghitung jeda waktu antar peluruhan. Tetapi, disini dibatasi sudut emisinya sehingga semua partikel di dalam simulasi menabrak detektor satu persatu, dimana ini akan lama visualisasinya.
- **Masalah:** Kode menghitung jeda waktu seolah-olah 1000 partikel alfa *secara keseluruhan* memakan waktu sekitar 1 detik (karena aktivitas 1000 Bq). Kenyataannya, efisiensi geometris (*solid angle*) detektormu adalah:

Hasil Uji Algoritma:

Dalam pengoperasiannya, sebagian besar operasi bersifat vectorized menggunakan Numpy. Kelebihannya yang sangat efisien di Python dapat menjadikan sampling energi, sampling arah, sampling waktu, transformasi koordinat menjadi efisien. Selain itu juga digunakan bentuk `np.random.rand(N_PARTICLES)` yang lebih cepat dibandingkan pemakaian loop Python biasa.

Visualisasi Awal:



Dipakai bentuk 100 partikel untuk visualisasi awal, yang tidak memakan waktu lama jika dibandingkan memakai 1000 partikel. Dari bentuk 100 partikel ini diperlukan waktu sekitar 20 - 35 detik untuk memvisualisasikan semua partikel.

Analisis Bottleneck Komputasi

Jika ditambahkan jumlah partikel (`N_PARTICLES`) menjadi 10,000 atau 50,000, program ini akan menjadi sangat lambat. Bottleneck program ini terbagi jelas secara diametral yang terletak di bagian rendering Loop di Matplotlib 3D.

Bottleneck 99% terjadi pada **Fungsi Animasi** `FuncAnimation` dan eksekusinya:

- Di dalam fungsi `update()`, dimodifikasi bentuk properti 3D menggunakan `.set_3d_properties()` pada ribuan objek `lines` dan `scatters` yang terpisah secara individual.
- Dapat dipahami bahwa Matplotlib 3D tidak dioptimasi untuk merender ribuan elemen *artist* secara paralel (Matplotlib tidak menggunakan akselerasi GPU murni layaknya OpenGL).
- Menyimpan *list of objects* seperti `lines = [ax.plot(...)[0] for _ in range(N_PARTICLES)]` menciptakan *overhead* memori yang masif. Saat frame bertambah, Matplotlib harus melakukan iterasi pada *artist list* yang sangat bengkak di setiap iterasi layar (interval=200ms).